PROJECT REPORT

AI Powered Emergency Resource Allocation and Decision Support System

ReliefPlan AI

16 October 2026

GitHub Repository

https://github.com/palli-pavan/Disaster

Abstract

ReliefPlan AI is a disaster response project that estimates resource needs and recommends how available supplies can be distributed across affected areas. It uses damage observations, population records, road information and warehouse stock. A trained XGBoost model predicts damage severity, and a linear programming model allocates food, water and medical kits within stock and demand limits.

The application provides a four stage workflow for describing a disaster, reviewing AI analysis, inspecting a response plan and testing changes in supply. A map displays the study areas and their priority levels. The project also includes SHAP explanations, scenario comparison and a CSV decision history.

The default core run estimated 12,846 affected residents across ten areas. It allocated 15,000 food rations, 20,000 litres of water and 1,927 medical kits. A 30 percent reduction in water stock lowered water coverage from 17.30 percent to 12.11 percent. All 24 existing automated tests passed. The results demonstrate the processing pipeline on synthetic data; the current application remains a prototype with several interface and allocation policy limitations.

Introduction

Problem Statement

During a disaster, emergency teams must distribute limited supplies among communities with different levels of damage and need. Population alone does not describe the urgency of each area. Structural damage, vulnerable age groups and blocked roads also affect response planning. A useful decision support system should bring these factors together and make its recommendations understandable.

Project Objectives

- Predict damage severity from structural damage and disaster characteristics.
- Estimate the number of residents needing assistance and calculate resource demand.
- Assign area priorities using damage, demand, demographic and accessibility information.
- Allocate available food, water and medical supplies without exceeding stock or demand.
- Display recommendations on a dashboard and compare changes in supply or population.

Scope of the Project

The project uses synthetic records for a Houston study area. The core pipeline processes ten areas, A to J. The React API restricts its analysis to seven areas, A to G, and accepts population values from the interface. The two analysis paths therefore produce different results.

The recommendations support a human decision maker. The application does not control vehicles, confirm delivery or send public emergency messages. Hospital and shelter data are supplied, but separate bed and shelter assignment are outside the current core pipeline.

Repository

The project source is available at the following address. The report describes the implementation and outputs from the supplied Disaster folder.

https://github.com/palli-pavan/Disaster

System Design

The computation is kept in reusable Python modules. React communicates with these modules through FastAPI, while the Streamlit interface calls the Python pipeline directly. CSV files hold the input data and decision history; trained models are loaded from the models directory.

Processing Flow

```
Input CSV files and saved model
    -> Validate and merge data
    -> Predict damage severity
    -> Estimate affected population and demand
    -> Calculate priority scores
    -> Solve resource allocation
    -> Generate actions and warnings
    -> Display results and record the decision
```

Component	Technology	Purpose
Frontend	React and Vite	Scenario inputs and four stage dashboard
API	FastAPI and Pydantic	Validate requests and return analysis results
Data processing	Pandas and NumPy	Read, join and prepare tabular data
Prediction	Scikit learn and XGBoost	Train and compare severity classifiers
Optimization	SciPy HiGHS	Solve bounded allocation linear programs
Explainability	SHAP	Explain classifier outputs
Map	Leaflet and OpenStreetMap	Display area locations and risk levels
Alternative interface	Streamlit	Run the guided Python dashboard

Source Organization

The core directory contains data preparation, prediction, demand, priority, optimization, explanation, simulation and audit modules. api.py exposes the backend handlers. frontend/src contains the React application and styles. app/Home.py provides the Streamlit interface. The tests directory contains five automated test modules. Earlier modules remain in src and app/_pages_archive.

Dataset Description

File	Rows	Description
damage.csv	10	Structural damage observations
population.csv	10	Population, coordinates and vulnerable age groups
disasters.csv	3	Event type, magnitude and event details
resources.csv	4	Available resource quantity by warehouse
warehouses.csv	2	Depot identifiers and locations
roads.csv	22	Area connections and road conditions
hospitals.csv	2	Hospital locations and capacity
shelters.csv	2	Shelter capacity and occupancy
scenarios.csv	3	Named example scenarios
historical_disaster_damage.csv	800	Labelled training examples

Feature Preparation

Damage observations are joined with population records using area_id and disaster information using disaster_id. Missing damage counts are filled with zero. If building_damage is unavailable, it is reconstructed from minor, major and destroyed building counts. Missing population density and magnitude are filled with medians, with fixed fallback values when necessary.

Classifier input	Description
building_damage	Total damaged buildings
minor_damage	Buildings with minor damage
major_damage	Buildings with major damage
destroyed_buildings	Buildings classified as destroyed
population_density	Population density of the area
magnitude	Magnitude recorded for the disaster

The target classes are LOW, MEDIUM, HIGH and CRITICAL. The historical dataset contains 800 synthetic examples. The saved evaluation uses 640 records for training and 160 for the held out test set.

Implementation and Setup

Environment Setup

Run the following commands from the emergency-resource-allocation directory. FastAPI and Uvicorn must be installed separately because they are not listed in the supplied requirements file.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pip install fastapi uvicorn
```

Model Artifacts

The project already includes best_model.pkl, damage_model.pkl, label_encoder.pkl, shap_explainer.pkl and metrics.json. The captured results use these artifacts. To train the three candidate models again, run the command below. Retraining replaces the model files and evaluation results.

```
python core/model.py
```

Run the Backend and Frontend

Start the API from the project directory. In a second terminal, open the frontend directory and start Vite.

```
python -m uvicorn api:app --host 127.0.0.1 --port 8000

# Second terminal
cd frontend
npm ci
npm run dev -- --host 127.0.0.1
```

Open the local URL printed by Vite, normally http://127.0.0.1:5173. The React application calls the API at http://127.0.0.1:8000.

Testing and the Streamlit Interface

```
python -m pytest tests -v
streamlit run app/Home.py
```

Streamlit normally opens at http://localhost:8501. The Python modules use paths relative to the project directory, so commands should be run from that directory.

Processing Methodology

Data Loading

load_all_data reads the configured CSV files. validate_dataset checks required columns and reports empty tables or negative values. prepare_ml_dataset merges the main tables and prepares the six classifier inputs.

Severity Prediction

The training module compares Logistic Regression, Random Forest and XGBoost using a stratified 80 to 20 split and five fold cross validation. The implementation selects the model with the highest held out macro F1. During analysis, it loads the saved model and decodes predicted class labels.

Demand Estimation

The predicted severity determines the affected population ratio. Resource demand is calculated from affected population, daily consumption rate and response duration. Duration supplied in hours is converted to days by the API.

Priority Calculation

The priority module combines structural impact, demand, a shortage estimate, vulnerable population and road access obstruction. Component values and the final score are retained so that the result can be inspected.

Resource Allocation

A linear program is solved for each resource. Total allocation is limited by available stock, while each area is capped at its own demand. A population proportional allocation is calculated as the comparison baseline.

Explanation and Output

The system creates action cards and warnings from allocation results. The explanation module calculates local and global SHAP values and shows the components of a priority score. The API returns the results as JSON for the React dashboard.

Simulation and Audit

The pipeline can be rerun after supply or population changes. The analysis handler writes a CSV record with a timestamp, scenario, model, solver details and allocation totals. Calling the core pipeline alone does not automatically write an audit record.

Demand and Priority Calculations

Affected Population and Demand

```
affected = round(population * severity_ratio)
demand = affected * daily_resource_rate * duration_days
shortage = max(0, demand - allocation)
coverage = min(1, total_supply / total_demand)
```

Severity	Affected population ratio
LOW	0.10
MEDIUM	0.30
HIGH	0.55
CRITICAL	0.85

The default core rates are 2 food rations, 3 litres of water and 0.05 medical kits per affected person per day, for three days. These rates and the severity ratios are project assumptions. The API uses different rates: food remains 2, water is 2 or 2.5 for Wildfire, and medical kits are 0.12, 0.18 for Wildfire or 0.25 for Earthquake.

Priority Score

```
priority = 0.25 * normalized_impact
         + 0.25 * normalized_demand
         + 0.20 * normalized_shortage
         + 0.15 * normalized_medical_need
         + 0.15 * access_penalty
```

Impact combines building damage with a severity weight. Medical need is calculated as 1.5 times elderly population plus 1.2 times children population. If total_shortage is unavailable, the shortage input is half of estimated demand. The first four components use min max normalization. Road obstruction is already bounded between zero and one, using OPEN = 0, DAMAGED = 0.5 and BLOCKED = 1.

Scores below 0.25 are LOW, scores from 0.25 to below 0.50 are MEDIUM, scores from 0.50 to below 0.75 are HIGH, and scores of 0.75 or more are CRITICAL. Operational priority is distinct from the severity class predicted by the classifier.

Resource Allocation Model

For one resource, x_i is the quantity allocated to area i, D_i is its demand, S is total stock, and p_i is the priority score with a minimum coefficient of 0.05. The allocation problem is:

```
Maximize   sum_i (p_i / D_i) * x_i

Subject to sum_i x_i <= S
           0 <= x_i <= D_i
```

SciPy linprog minimizes the negative of this objective using the HiGHS solver. The model rewards priority weighted demand satisfaction. Since benefit is divided by demand, an area with the highest priority score does not necessarily receive the first allocation. A positive objective coefficient does not guarantee a positive allocation.

Baseline Allocation

```
baseline_i = min(D_i, S * population_i / total_population)
reduction = max(0, baseline_shortage - optimized_shortage)
reduction_percent = 100 * reduction / baseline_shortage
```

The comparison baseline distributes stock in proportion to resident population and caps the result at demand. In the default ten area experiment, both methods use the available food and water stock, and the reported total shortage reduction is zero. Allocation can change without reducing total shortage when the same scarce stock is fully consumed.

Warehouse Allocation

A separate transportation LP supports warehouse to area quantities and includes a small great circle distance penalty. It checks depot stock and area demand. The default core pipeline does not call this routine, and blocked road paths are not constraints in this transportation model.

Units and Rounding

Food is measured in rations, water in litres and medical supplies in kits. The core rounds quantities to one decimal place; the API rounds displayed area values to integers. The program also reports a combined shortage KPI that adds different resource units. The resource specific results below are used for interpretation.

Results of the Core Analysis

The baseline uses all ten areas, the supplied model artifacts, the original warehouse quantities and a three day response duration. No population or supply shock is applied.

Measure	Result
Resident population	30,260
Estimated affected residents	12,846
Areas with CRITICAL priority	1
Areas with HIGH priority	1
Food solver	OPTIMAL
Water solver	OPTIMAL
Medical kit solver	OPTIMAL

Resource	Demand	Supply	Allocated	Shortage	Coverage
Food	77,076.0	15,000.0	15,000.0	62,076.0	19.46%
Water	115,614.0	20,000.0	20,000.0	95,614.0	17.30%
Medical Kit	1,927.0	5,000.0	1,927.0	0.0	100.00%

Food and water stock are fully allocated, but they cover only a small part of estimated demand. Medical stock exceeds demand, leaving 3,073 kits unused. All three resource problems return OPTIMAL for this experiment. The reported shortage reduction against the population baseline is 0.00 percent.

Example Calculation for Area A

Area A has 2,368 residents and predicted MEDIUM severity. Its affected population is round(2368 × 0.30) = 710. Water demand is 710 × 3 × 3 = 6,390 litres. The optimizer allocates 6,390 litres, giving zero water shortage for Area A.

The complete values are retained in baseline_summary.json and baseline_allocations.csv in the evidence folder.

Area Allocation Results

Area	Predicted severity	Priority	Priority level	Affected
E	HIGH	0.7858	CRITICAL	2,552
H	HIGH	0.7074	HIGH	2,417
C	HIGH	0.4386	MEDIUM	1,036
J	HIGH	0.4336	MEDIUM	1,114
I	HIGH	0.4299	MEDIUM	1,073
B	MEDIUM	0.3362	MEDIUM	1,336
D	HIGH	0.3345	MEDIUM	1,575
A	MEDIUM	0.3118	MEDIUM	710
F	HIGH	0.2772	MEDIUM	568
G	LOW	0.2086	LOW	465

Area	Food rations	Water litres	Medical kits	Water shortage
A	4,260.0	6,390.0	106.5	0.0
B	0.0	0.0	200.4	12,024.0
C	4,542.0	4,313.0	155.4	5,011.0
D	0.0	0.0	236.2	14,175.0
E	0.0	0.0	382.8	22,968.0
F	3,408.0	5,112.0	85.2	0.0
G	2,790.0	4,185.0	69.8	0.0
H	0.0	0.0	362.6	21,753.0
I	0.0	0.0	161.0	9,657.0
J	0.0	0.0	167.1	10,026.0

Area E has the highest priority score, 0.7858, but receives zero food and water in this run. This follows from the priority divided by demand objective and the absence of a minimum allocation constraint. The result shows why allocation policy must be reviewed alongside solver status.

One generated recommendation sends 4,313 litres of water to Area C. Its water demand is 9,324 litres, leaving 5,011 litres unmet after allocation. Recommendation quantities come from the optimizer; they do not represent confirmed deliveries.

Model Evaluation

The following results are recorded in the supplied models/metrics.json. They are the saved training evaluation, rather than a new training run. Macro F1 is the selection metric.

Model	Accuracy	Macro F1	CV macro F1	ROC AUC
Logistic Regression Baseline	67.50%	0.6765	0.6367	0.9338
Random Forest	78.75%	0.7183	0.6697	0.9464
XGBoost	81.87%	0.7366	0.6634	0.9573

XGBoost has the highest saved held out macro F1, 0.7366, and recorded accuracy of 81.87 percent. Random Forest has the highest mean cross validation macro F1, 0.6697. The current selection uses the held out result, so it should not be described as a selection based on cross validation.

XGBoost Confusion Matrix

Rows show actual classes and columns show predicted classes.

Actual	CRITICAL	HIGH	LOW	MEDIUM
CRITICAL	8	9	0	0
HIGH	4	88	0	7
LOW	0	0	7	4
MEDIUM	0	4	1	28

Local SHAP Output for Area A

Feature	Input	SHAP value
destroyed_buildings	26	+1.1028
population_density	1,108	+0.3637
minor_damage	69	+0.0935
magnitude	7.5	+0.0578
major_damage	138	-0.0401
building_damage	233	-0.0200

Area A is predicted as MEDIUM. Positive SHAP values support this predicted class; they do not establish physical causes or an ordinal increase in disaster risk. Contributions describe model behaviour, not causation.

Scenario Analysis

Two separate changes were tested against the default core run: a 30 percent reduction in water supply and a 25 percent increase in population. Other settings were held at the core defaults for each experiment.

Resource	Baseline coverage	Water shock coverage
Food	19.46%	19.46%
Water	17.30%	12.11%
Medical Kit	100.00%	100.00%

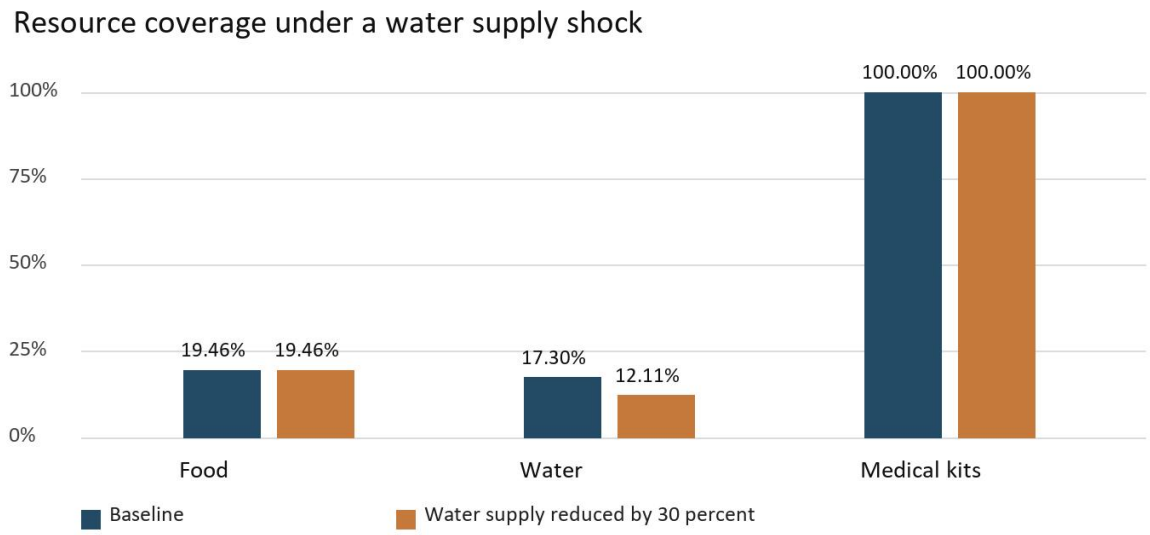


Figure 1 Resource coverage before and after the water supply reduction

Water measure	Baseline	After shock
Supply in litres	20,000	14,000
Demand in litres	115,614	115,614
Shortage in litres	95,614	101,614
Affected residents	12,846	12,846

The water shock increases unmet water demand by 6,000 litres. In the separate population experiment, estimated affected population increases to 16,058. Food coverage falls to 15.57 percent and water coverage to 13.84 percent.

Dashboard for Disaster Inputs

The Describe screen collects disaster type, severity, response duration, available supplies and population values for seven areas. The captured scenario uses Wildfire, High severity and 72 hours. The displayed area populations total 21,779.

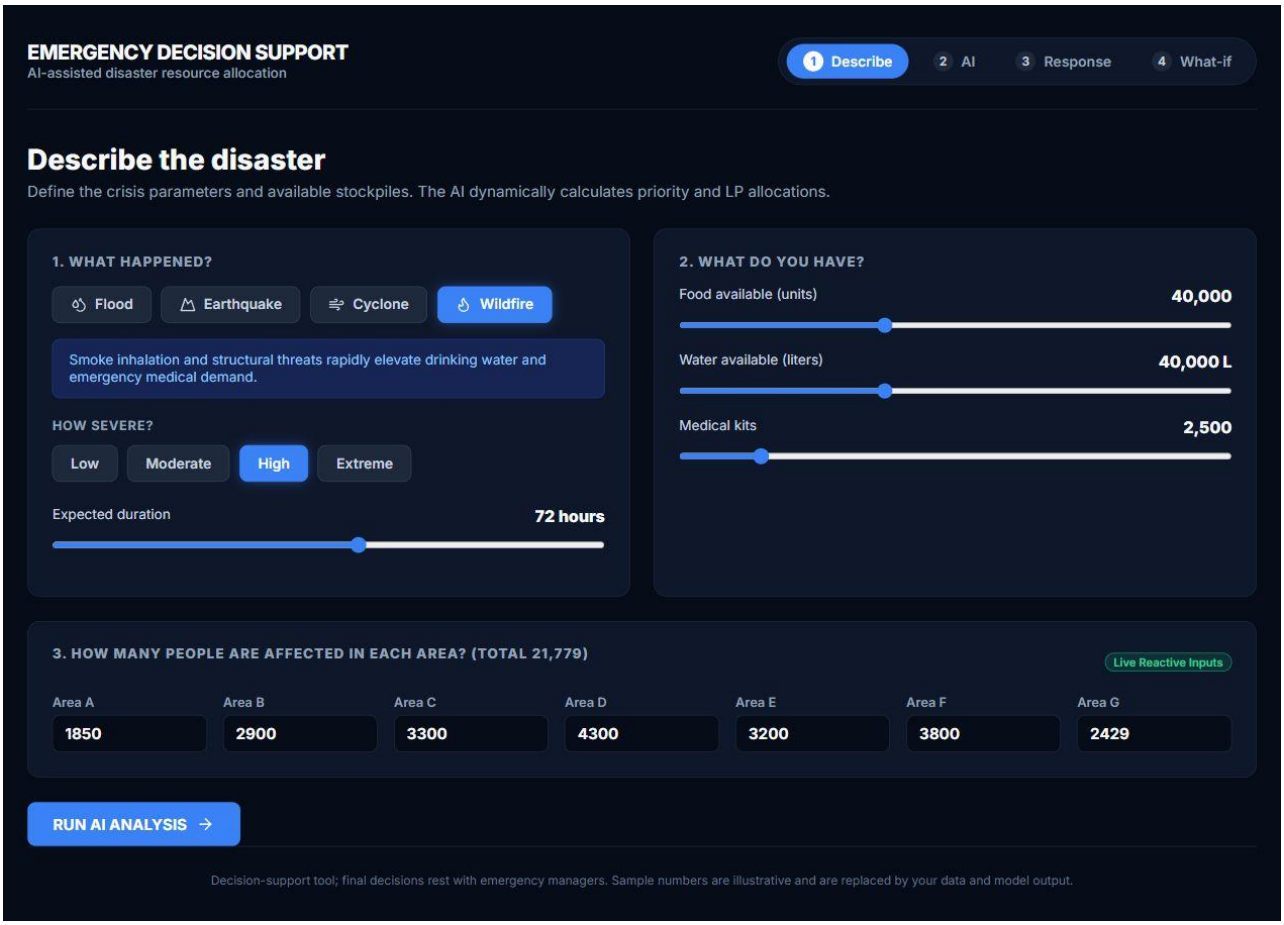


Figure 2 Disaster description and resource input screen

Input Handling

Population edits are sent to the analysis API, which updates the seven study areas before running the pipeline. Hours are converted into days. The UI displays food, water and medical stock inputs, but the current API writes these to total_quantity while the optimizer reads quantity. The displayed stock inputs therefore do not change the stock used by allocation.

The population values in this screen differ from the Pydantic request defaults and from the original ten area CSV data. Dashboard figures should be interpreted using the scenario shown in the screenshot.

Dashboard for AI Analysis

The AI analysis screen combines summary values, the geographical map and the area ranking. The captured run shows 21,779 residents, no CRITICAL priority areas, four HIGH priority areas and 28 percent water coverage.

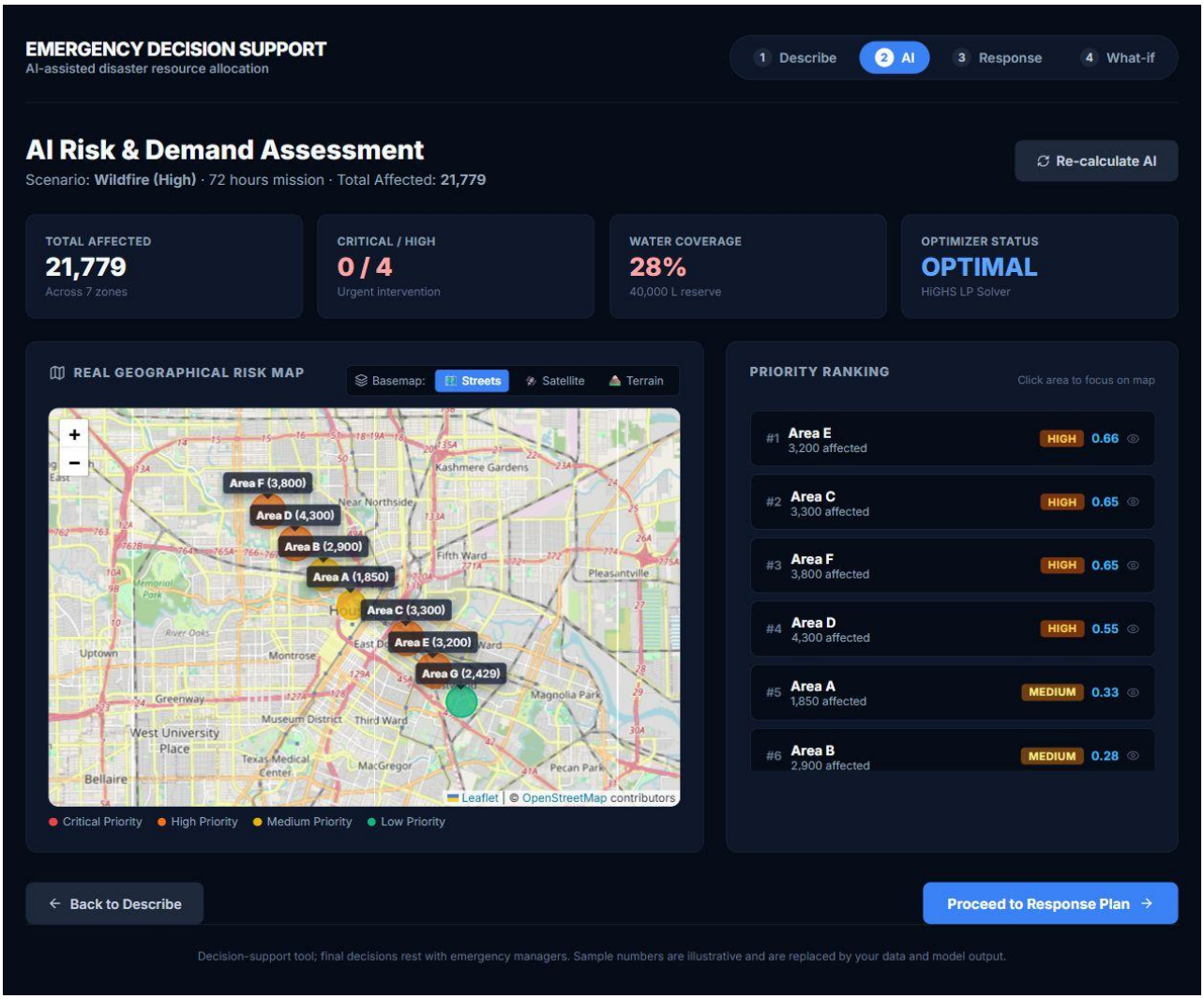


Figure 3 AI analysis dashboard with the Houston risk map and area ranking

Reading the Results

Area E ranks first with a displayed priority score of 0.66, followed by Areas C and F at 0.65. Markers use the area priority colour and can be selected to inspect a location. The map uses Leaflet with OpenStreetMap tiles.

The dashboard label TOTAL AFFECTED currently sums resident population. It is not the severity adjusted estimated_affected value. The stock reserve label repeats the interface input, while the coverage calculation uses the CSV stock quantity.

Dashboard for the Response Plan

The response screen turns allocations into recommendation cards. Each card shows the resource quantity, the reason for its priority and the remaining shortage after allocation.

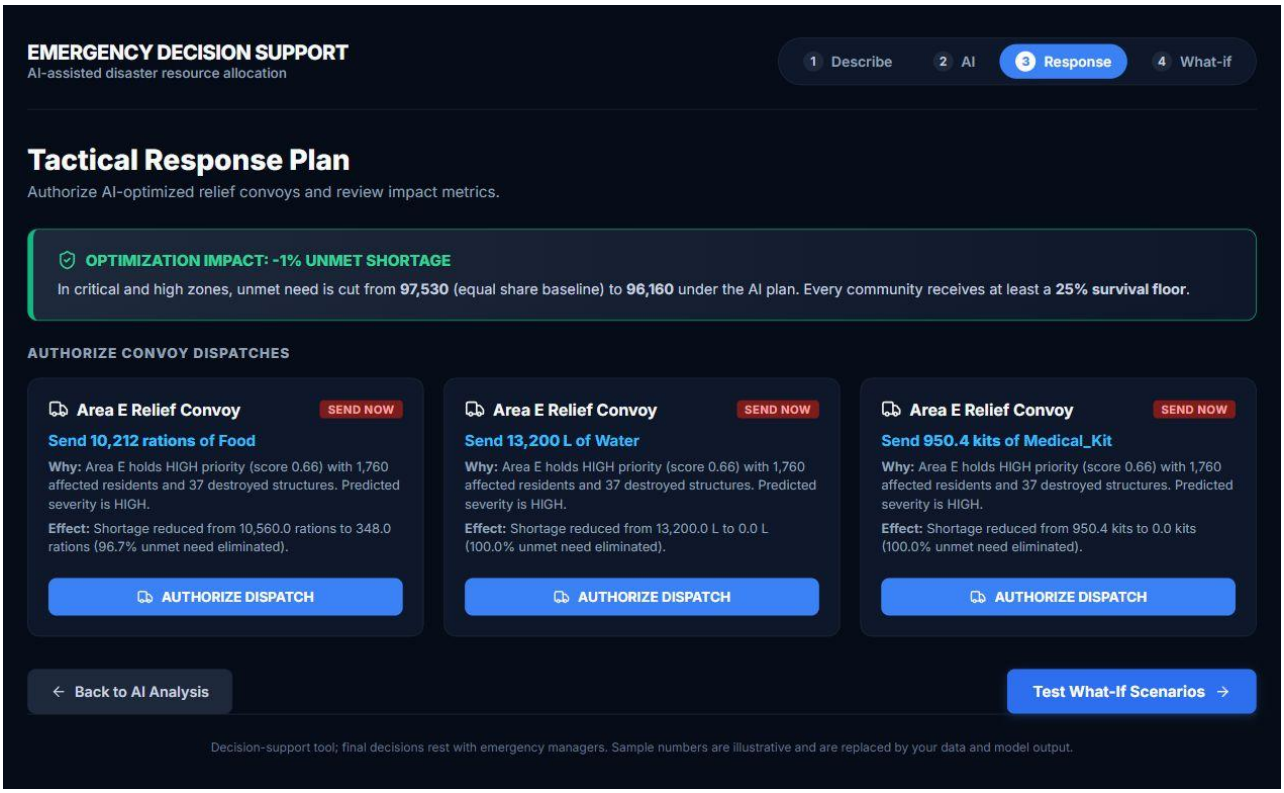


Figure 4 Response plan with the generated resource recommendation cards

Example Recommendation

For the captured React scenario, the first Area E card recommends 10,212 food rations against demand of 10,560. It reports 348 rations still unmet. The following cards show 13,200 litres of water and 950.4 medical kits for the same area.

The 25 percent survival floor statement visible in the screen is not enforced by the current LP. The allocation lower bound is zero. The impact banner also uses the aggregate comparison returned by the backend, although its wording refers to critical and high zones. These screen statements are implementation gaps, not guarantees established by the results.

AUTHORIZE DISPATCH changes the local interface state only. There is no connected convoy control or delivery confirmation service.

Dashboard for Simulation and Audit History

The What if screen tests supply and population changes and displays the decision history. A 30 percent water reduction was executed before capturing the screen.

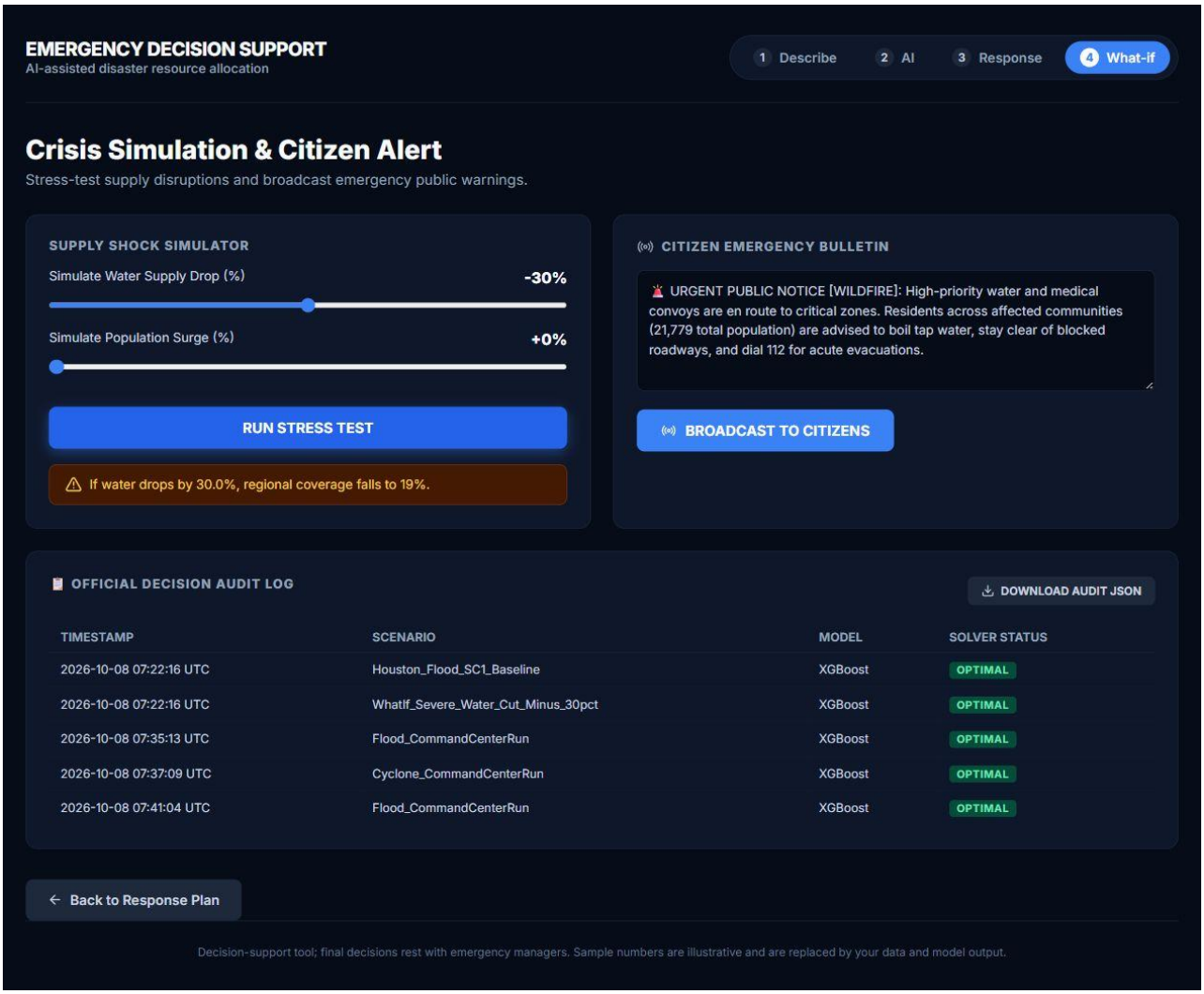


Figure 5 Water supply simulation result and decision audit history

Simulation Output

The API simulation reports water coverage falling from 27 percent to 19 percent, a decrease of eight displayed percentage points. This handler reloads seven areas with CSV populations and core default rates. It does not carry forward the Describe screen inputs, so it has a different baseline from the 28 percent coverage shown in the AI analysis screenshot.

The audit table displays existing timestamped scenario records. The bulletin text and broadcast button are interface demonstrations; clicking the button does not transmit an alert to citizens.

Testing and Verification

Check	Result
Existing Python tests	24 passed with one pandas FutureWarning
React production build	Completed with Vite 8.3.2
Core allocation	All three resource solvers returned OPTIMAL
Water shock	Water shortage increased by 6000 litres
Population surge	Demand increased and the pipeline completed
Warehouse routine	OPTIMAL with 20000 litres shipped from W1
Dashboard workflow	Four screens opened and water stress test completed
API stock comparison	Inputs reduced to 1 left allocations unchanged

Automated Test Coverage

Test module	Tests	Main checks
test_actions_and_explain.py	5	Action quantities, reasons, SHAP and score components
test_demand.py	4	Demand arithmetic, bounds and zero population
test_optimizer.py	7	Stock and demand limits, solver output and transport
test_priority.py	4	Bounds, categories, determinism and normalization
test_simulate_and_audit.py	4	Supply shock, comparison and audit persistence

The warning comes from filling missing values in an object array in core/explain.py. It concerns future pandas behaviour and did not fail a test. The tests do not cover every interface statement or API input connection.

Execution Environment

Verification used Python 3.13.7 on Windows 11. Main versions were Pandas 2.3.3, NumPy 2.2.6, SciPy 1.16.2, XGBoost 3.2.0, SHAP 0.50.0 and FastAPI 0.135.1.

The production build transformed 1,938 modules. API handlers were also executed directly with Pydantic requests. The dashboard screenshots show a separate browser run of the React workflow.

Limitations and Future Work

Synthetic Data

Damage and population records are generated examples. The model has not been validated on independently collected operational disaster data.

Stock Input Connection

The API must update the quantity field consumed by the optimizer. Requested totals also need a defined distribution across warehouse rows.

Minimum Service Policy

The current allocation can leave an area with zero supply. Any minimum service policy must be written as a feasible LP constraint and tested under severe scarcity.

Scenario Consistency

The simulation handler reloads a separate baseline. It should receive the current analysis inputs so its before and after results describe the same scenario.

Status and Population Labels

The API defaults a missing singular solver_status field to OPTIMAL. It should return actual per resource statuses and is_optimal. The affected population label should be aligned with estimated_affected.

Prediction Evaluation

The current model selection uses the held out test metric. A separate validation procedure would leave the final test set independent of selection.

Audit and Routing

CSV logs can be edited or cleared, so they are not tamper proof. The transportation LP penalizes distance but does not enforce a road network route around blocked links.

Operational Integrations

Dispatch and broadcast actions are local state changes. A deployed system would need confirmed resource movement, authenticated users and connected communication services.

Conclusion and Source Reference

ReliefPlan AI implements the main stages of an emergency resource planning workflow: data preparation, severity prediction, demand estimation, priority scoring, constrained allocation and scenario comparison. The dashboards provide a way to inspect the inputs, locate affected areas and review recommended quantities.

The tests and captured runs show that the processing modules execute with the supplied synthetic data and saved model artifacts. The water supply experiment illustrates the effect of reduced stock on resource coverage. The area results also show that an optimal solution depends on the chosen objective and policy constraints; solver status alone does not establish an acceptable distribution of aid.

The next development work should connect stock inputs correctly, keep simulation inputs consistent with the current analysis and add a clear minimum service policy. The interface labels and displayed guarantees should then be checked against the computed results. Validation on independent disaster records is needed before operational use.

GitHub Repository

https://github.com/palli-pavan/Disaster

Files Included with the Report

The accompanying submission package contains the complete Python and React source, the input CSV files, trained model artifacts, tests, captured output and the four original dashboard images. The code appendix contains selected functions without printed line numbers.

Reproducing the Output

After extracting the package, install the project dependencies and run the tests from emergency-resource-allocation. To reproduce the JSON and CSV output, run python evidence/capture_outputs.py from the extracted package root. New verification audit records are kept separately from the project audit history.

Source Files Used for the Results

Baseline allocations and supply shock results come from core/simulate.py, core/optimizer.py and the packaged evidence CSV and JSON files. Model evaluation values come from models/metrics.json. Dashboard screenshots were captured from the running React frontend connected to api.py.

Data Preparation Code

Source file core/data.py

```
def prepare_ml_dataset(datasets: Dict[str, pd.DataFrame]) -> pd.DataFrame:
    """Merges damage, population, and disaster tables for model training or inference.

    Args:
        datasets: Dictionary of DataFrames as returned by load_all_data.

    Returns:
        Consolidated DataFrame with engineered features ready for ML ingestion.
    """
    if "damage" not in datasets or "population" not in datasets or "disasters" not in datasets:
        raise ValueError("Cannot prepare ML dataset: 'damage', 'population', or 'disasters'
missing.")

    # Start from damage table
    df = datasets["damage"].copy()

    # Join population attributes by area_id
    pop_df = datasets["population"]
    df = pd.merge(df, pop_df, on="area_id", how="left")

    # Join disaster magnitude and characteristics by disaster_id
    disaster_df = datasets["disasters"]
    df = pd.merge(df, disaster_df, on="disaster_id", how="left")

    # Missing value imputation with safe defaults
    df["minor_damage"] = df["minor_damage"].fillna(0)
    df["major_damage"] = df["major_damage"].fillna(0)
    df["destroyed_buildings"] = df["destroyed_buildings"].fillna(0)

    if "building_damage" not in df.columns or df["building_damage"].isnull().all():
        df["building_damage"] = df["minor_damage"] + df["major_damage"] +
df["destroyed_buildings"]
    else:
        df["building_damage"] = df["building_damage"].fillna(
            df["minor_damage"] + df["major_damage"] + df["destroyed_buildings"]
        )

    median_density = df["population_density"].median() if not
df["population_density"].dropna().empty else 3000.0
    df["population_density"] = df["population_density"].fillna(median_density)

    median_mag = df["magnitude"].median() if not df["magnitude"].dropna().empty else 6.0
    df["magnitude"] = df["magnitude"].fillna(median_mag)

    return df
```

Affected Population Code

Source file core/demand.py

```
def estimate_affected_population(
    population: float,
    severity: str,
    custom_ratios: Optional[Dict[str, float]] = None
) -> int:
    """Estimates the acute affected population requiring aid in an area.

    Args:
        population: Total resident population in the area.
        severity: Assessed or predicted damage severity level.
        custom_ratios: Optional override for severity affected ratios.

    Returns:
        Integer count of affected residents.
    """
    ratios = custom_ratios or SEVERITY_AFFECTED_RATIOS
    ratio = ratios.get(str(severity).upper(), 0.30)
    return int(max(0, round(population * ratio)))
```

Resource Demand Code

Source file core/demand.py

```
def compute_area_demands(
    areas_df: pd.DataFrame,
    per_capita_needs: Optional[Dict[str, float]] = None,
    duration_days: float = DEFAULT_DURATION_DAYS,
    use_affected_only: bool = True
) -> pd.DataFrame:
    """Computes exact demand for each resource type across all areas.

    Formula: demand_ik = population_i * per_capita_need_k * duration

    Args:
        areas_df: DataFrame containing area records with population and severity.
        per_capita_needs: Resource consumption standards per person per day.
        duration_days: Mission duration in days.
        use_affected_only: If True, uses affected population rather than total.

    Returns:
        DataFrame containing area identifier, population, and demand columns:
        'demand_Food', 'demand_Water', 'demand_Medical_Kit'.
    """
    df = areas_df.copy()
    needs = per_capita_needs or DEFAULT_PER_CAPITA_NEEDS
    duration = max(0.1, float(duration_days))

    # Ensure estimated affected population is computed
    if "estimated_affected" not in df.columns:
        sev_col = "predicted_severity" if "predicted_severity" in df.columns else
"damage_severity"
        df["estimated_affected"] = df.apply(
            lambda r: estimate_affected_population(r.get("population", 1000), r.get(sev_col,
"MEDIUM")),
            axis=1
        )

    base_pop = df["estimated_affected"] if use_affected_only else df["population"]

    for resource, rate in needs.items():
        col_name = f"demand_{resource}"
        df[col_name] = (base_pop * float(rate) * duration).round(1)

    return df
```

Priority Calculation Code

Source file core/priority.py

```
def calculate_area_priorities(
    areas_df: pd.DataFrame,
    roads_df: Optional[pd.DataFrame] = None,
    weights: Optional[Dict[str, float]] = None,
    thresholds: Optional[Dict[str, float]] = None,
) -> pd.DataFrame:
    """Calculates normalized priority scores and risk tiers for all areas.

    Args:
        areas_df: DataFrame containing area data, damage metrics, population,
                  and predicted_severity.
        roads_df: Optional DataFrame of roads for accessibility analysis.
        weights: Dictionary of component weights.
        thresholds: Dictionary of risk classification thresholds.

    Returns:
        DataFrame enriched with 'priority_score', 'priority_level',
        and decomposition columns.
    """
    df = areas_df.copy()
    w = weights or DEFAULT_WEIGHTS
    th = thresholds or DEFAULT_THRESHOLDS

    # Normalize weights so they sum to 1.0
    w_sum = sum(w.values()) if sum(w.values()) > 0 else 1.0
    norm_w = {k: v / w_sum for k, v in w.items()}

    # 1. Impact: structural damage modulated by predicted severity
    sev_weights = df["predicted_severity"].map(SEVERITY_WEIGHT_MAP).fillna(0.3)
    bldg_dmg = df["building_damage"].fillna(0) if "building_damage" in df else pd.Series(0,
index=df.index)
    impact_raw = bldg_dmg * sev_weights + sev_weights * 100.0

    # 2. Predicted Demand: proportional to affected population
    if "estimated_affected" in df.columns:
        demand_raw = df["estimated_affected"].fillna(0)
    elif "population" in df.columns:
        demand_raw = df["population"] * sev_weights
    else:
        demand_raw = pd.Series(100.0, index=df.index)

    # 3. Shortage: unmet need estimate (if available)
    if "total_shortage" in df.columns:
        shortage_raw = df["total_shortage"].fillna(0)
    else:
        shortage_raw = demand_raw * 0.5  # Initial baseline proxy

    # 4. Medical Need: vulnerable demographic groups + injured
    elderly = df["elderly_population"].fillna(0) if "elderly_population" in df.columns else 0
    children = df["children_population"].fillna(0) if "children_population" in df.columns else 0
    medical_raw = elderly * 1.5 + children * 1.2

    # 5. Accessibility Penalty
    areas_list = df["area_id"].astype(str).tolist()
    penalties_dict = compute_accessibility_penalties(areas_list, roads_df)
    access_raw = df["area_id"].astype(str).map(penalties_dict).fillna(0.0)
```

Priority Calculation Code Continued

Source file core/priority.py

```
    # Normalization inside current scenario
    norm_impact = min_max_normalize(impact_raw)
    norm_demand = min_max_normalize(demand_raw)
    norm_shortage = min_max_normalize(shortage_raw)
    norm_medical = min_max_normalize(medical_raw)
    norm_access = access_raw.clip(0.0, 1.0)  # already bounded [0, 1]

    # Weighted Priority Score
    priority_score = (
        norm_w["impact"] * norm_impact +
        norm_w["demand"] * norm_demand +
        norm_w["shortage"] * norm_shortage +
        norm_w["medical_need"] * norm_medical +
        norm_w["accessibility_penalty"] * norm_access
    ).clip(0.0, 1.0)

    # Risk level classification
    priority_level = priority_score.apply(lambda s: classify_risk_level(s, th))

    # Add results and component breakdowns
    df["priority_score"] = priority_score.round(4)
    df["priority_level"] = priority_level
    df["norm_impact"] = norm_impact.round(4)
    df["norm_demand"] = norm_demand.round(4)
    df["norm_shortage"] = norm_shortage.round(4)
    df["norm_medical_need"] = norm_medical.round(4)
    df["norm_accessibility_penalty"] = norm_access.round(4)

    # Weighted contributions for SHAP-style breakdown
    df["comp_impact"] = (norm_w["impact"] * norm_impact).round(4)
    df["comp_demand"] = (norm_w["demand"] * norm_demand).round(4)
    df["comp_shortage"] = (norm_w["shortage"] * norm_shortage).round(4)
    df["comp_medical_need"] = (norm_w["medical_need"] * norm_medical).round(4)
    df["comp_accessibility"] = (norm_w["accessibility_penalty"] * norm_access).round(4)

    return df
```

Resource Allocation Code

Source file core/optimizer.py

```
def optimize_resource_allocation(
    areas_df: pd.DataFrame,
    resources_df: pd.DataFrame,
    resource_types: Optional[List[str]] = None
) -> Dict[str, Any]:
    """Solves the multi-resource priority-weighted Linear Program.

    Objective:
        Maximize sum_i (priority_i * x_ik / demand_ik)
        == Minimize sum_i (-priority_i / demand_ik * x_ik)

    Constraints:
        sum_i x_ik <= total_supply_k
        0 <= x_ik <= demand_ik for each area i

    Args:
        areas_df: DataFrame with 'area_id', 'priority_score', and 'demand_{resource}'.
        resources_df: DataFrame with 'resource_type' and 'quantity'.
        resource_types: Optional list of resource types to optimize.

    Returns:
        Dictionary containing:
            - 'allocation_df': enriched DataFrame with 'opt_alloc_{res}', 'opt_shortage_{res}'
            - 'solver_status': string summary per resource ('OPTIMAL', 'ZERO_SUPPLY', etc.)
            - 'metrics': objective values, runtime, shortage comparisons
    """
    start_time = time.perf_counter()
    df = areas_df.copy().reset_index(drop=True)
    n_areas = len(df)
    r_types = resource_types or list(DEFAULT_PER_CAPITA_NEEDS.keys())

    # Priority weights: ensure strictly positive so even low-risk gets proportional aid
    if "priority_score" in df.columns:
        p_weights = np.maximum(df["priority_score"].values.astype(float), 0.05)
    else:
        p_weights = np.ones(n_areas, dtype=float)

    solver_statuses: Dict[str, str] = {}
    is_all_optimal = True
    total_opt_shortage: float = 0.0
    total_initial_demand: float = 0.0

    for r_type in r_types:
        demand_col = f"demand_{r_type}"
        alloc_col = f"opt_alloc_{r_type}"
        shortage_col = f"opt_shortage_{r_type}"

        # Get total available supply
        if "resource_type" in resources_df.columns and "quantity" in resources_df.columns:
            tot_supply = float(resources_df[resources_df["resource_type"] ==
r_type]["quantity"].sum())
        else:
            tot_supply = 0.0
```

Resource Allocation Code Continued

Source file core/optimizer.py

```
        if demand_col not in df.columns:
            df[alloc_col] = 0.0
            df[shortage_col] = 0.0
            solver_statuses[r_type] = "NO_DEMAND_DATA"
            continue

        demands = df[demand_col].values.astype(float)
        total_initial_demand += float(demands.sum())

        # Handle zero supply case gracefully without solver failure
        if tot_supply <= 0:
            df[alloc_col] = 0.0
            df[shortage_col] = demands
            total_opt_shortage += float(demands.sum())
            solver_statuses[r_type] = "ZERO_SUPPLY (No stockpile available)"
            continue

        # Handle zero total demand case
        if demands.sum() <= 0:
            df[alloc_col] = 0.0
            df[shortage_col] = 0.0
            solver_statuses[r_type] = "ZERO_DEMAND (All areas satisfied)"
            continue

        # Build LP
        # c vector: minimize - (p_i / D_i) * x_i
        c = np.zeros(n_areas, dtype=float)
        for i in range(n_areas):
            if demands[i] > 0:
                c[i] = - (p_weights[i] / demands[i])
            else:
                c[i] = 0.0

        # Inequality constraint: sum_i x_i <= tot_supply
        A_ub = np.ones((1, n_areas), dtype=float)
        b_ub = np.array([tot_supply], dtype=float)

        # Bounds: 0 <= x_i <= demand_i
        bounds = [(0.0, float(demands[i])) for i in range(n_areas)]

        # Solve with SciPy Highs LP solver
        res = linprog(c=c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")

        if res.success and res.status == 0:
            solver_statuses[r_type] = "OPTIMAL"
            allocations = np.round(np.clip(res.x, 0.0, demands), 1)
        else:
            is_all_optimal = False
            solver_statuses[r_type] = f"INCOMPLETE ({res.message})"
            # Fallback to bounded proportional if LP failed
            prop = (demands / demands.sum()) * min(tot_supply, demands.sum())
            allocations = np.round(prop, 1)

        df[alloc_col] = allocations
        df[shortage_col] = np.maximum(0.0, demands - allocations).round(1)
        total_opt_shortage += float(df[shortage_col].sum())
```

Resource Allocation Code Continued

Source file core/optimizer.py

```
    elapsed_ms = round((time.perf_counter() - start_time) * 1000, 2)

    # Compute baseline for comparison
    df = compute_baseline_proportional_allocation(df, resources_df, r_types)

    # Compute comparative performance
    baseline_shortage_cols = [f"baseline_shortage_{r}" for r in r_types if
f"baseline_shortage_{r}" in df.columns]
    total_baseline_shortage = float(df[baseline_shortage_cols].sum().sum()) if
baseline_shortage_cols else total_initial_demand

    shortage_reduction = max(0.0, total_baseline_shortage - total_opt_shortage)
    reduction_pct = round((shortage_reduction / total_baseline_shortage) * 100.0, 2) if
total_baseline_shortage > 0 else 0.0

    return {
        "allocation_df": df,
        "is_optimal": is_all_optimal,
        "solver_statuses": solver_statuses,
        "execution_time_ms": elapsed_ms,
        "summary": {
            "total_initial_demand": round(total_initial_demand, 1),
            "total_baseline_shortage": round(total_baseline_shortage, 1),
            "total_optimized_shortage": round(total_opt_shortage, 1),
            "shortage_reduction_units": round(shortage_reduction, 1),
            "shortage_reduction_pct": reduction_pct,
            "overall_status": "OPTIMAL" if is_all_optimal else "PARTIAL_OR_UNCONSTRAINED"
        }
    }
```

Damage Prediction Code

Source file core/model.py

```
def predict_damage_severity(
    model: BaseEstimator,
    df_features: pd.DataFrame,
    label_encoder: Optional[LabelEncoder] = None,
    return_proba: bool = True
) -> Tuple[np.ndarray, Optional[np.ndarray], List[str]]:
    """Generates severity predictions and probabilities with fallback handling.

    Args:
        model: Trained classifier model.
        df_features: Input DataFrame containing required feature columns.
        label_encoder: Fitted LabelEncoder for class names, if applicable.
        return_proba: Whether to compute and return class probabilities.

    Returns:
        Tuple of (predicted_class_labels, probability_matrix_or_None, class_names).
    """
    X = df_features[ML_FEATURE_COLS].copy().fillna(0)
    raw_preds = model.predict(X)

    # Convert numeric predictions to class strings if encoder provided
    if label_encoder is not None:
        try:
            preds = label_encoder.inverse_transform(raw_preds)
            classes = list(label_encoder.classes_)
        except Exception:
            preds = raw_preds
            classes = list(getattr(model, "classes_", []))
    else:
        preds = raw_preds
        classes = list(getattr(model, "classes_", []))

    proba = None
    if return_proba and hasattr(model, "predict_proba"):
        try:
            proba = model.predict_proba(X)
        except Exception:
            proba = None

    return preds, proba, [str(c) for c in classes]
```

Decision Audit Code

Source file core/audit.py

```
def log_decision_run(
    scenario_name: str,
    pipeline_result: Dict[str, Any],
    model_name: str = "XGBoost",
    audit_file_path: str = AUDIT_LOG_CSV,
    additional_notes: str = ""
) -> Dict[str, Any]:
    """Records a complete execution run into the persistent audit trail.

    Args:
        scenario_name: Descriptive name of the disaster scenario.
        pipeline_result: Output dictionary from run_full_pipeline or
optimize_resource_allocation.
        model_name: Name of the active ML model.
        audit_file_path: Path to the audit log CSV.
        additional_notes: Operational comments or operator ID.

    Returns:
        Dictionary containing the logged record.
    """
    os.makedirs(os.path.dirname(audit_file_path), exist_ok=True)
    now_utc = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")
    run_id = f"RUN_{uuid.uuid4().hex[:8].upper()}"

    alloc_df = pipeline_result["allocation_df"]
    kpis = pipeline_result.get("kpis", {})
    solver_statuses = pipeline_result.get("solver_statuses", {})
    params = pipeline_result.get("parameters", {})

    # Priority scores summary
    p_scores = alloc_df["priority_score"].tolist() if "priority_score" in alloc_df else []
    avg_priority = round(float(pd.Series(p_scores).mean()), 3) if p_scores else 0.0

    # Severity prediction distribution
    sev_counts = alloc_df["predicted_severity"].value_counts().to_dict() \
        if "predicted_severity" in alloc_df else {}

    # Allocation breakdown by resource
    alloc_summary = {}
    for col in alloc_df.columns:
        if col.startswith("opt_alloc_"):
            r_type = col.replace("opt_alloc_", "")
            alloc_summary[r_type] = round(float(alloc_df[col].sum()), 1)

    # Status summary
    all_opt = pipeline_result.get("is_optimal", True)
    status_str = "OPTIMAL" if all_opt else "SUB_OPTIMAL"
```

Decision Audit Code Continued

Source file core/audit.py

```
    record: Dict[str, Any] = {
        "run_id": run_id,
        "timestamp": now_utc,
        "scenario_name": scenario_name,
        "model_name": model_name,
        "solver_status": status_str,
        "total_affected_pop": kpis.get("total_affected_population",
int(alloc_df.get("estimated_affected", pd.Series(0)).sum())),
        "critical_areas": kpis.get("critical_areas_count", int((alloc_df.get("priority_level",
pd.Series()) == "CRITICAL").sum())),
        "avg_priority_score": avg_priority,
        "total_shortage": kpis.get("total_shortage_units", 0.0),
        "shortage_reduction_pct": kpis.get("shortage_reduction_pct", 0.0),
        "dispatched_allocations": json.dumps(alloc_summary),
        "predictions_distribution": json.dumps(sev_counts),
        "solver_statuses_detail": json.dumps(solver_statuses),
        "scenario_inputs": json.dumps(params),
        "notes": additional_notes
    }

    df_new = pd.DataFrame([record])
    if os.path.exists(audit_file_path):
        df_new.to_csv(audit_file_path, mode="a", header=False, index=False)
    else:
        df_new.to_csv(audit_file_path, mode="w", header=True, index=False)

    return record
```

React Analysis Request Code

Source file frontend/src/App.jsx

```
const runAnalysis = async (navigateToStep2 = false) => {
  setLoading(true);
  setError(null);
  try {
    // Ensure all populations are clean integers
    const sanitizedPops = {};
    Object.entries(areaPops).forEach(([k, v]) => {
      sanitizedPops[k] = Math.max(1, parseInt(v, 10) || 100);
    });

    const res = await fetch(`${API_BASE}/api/analyze`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        disaster_type: disasterType,
        severity: severity,
        duration_hours: durationHours,
        food_supply: Number(foodSupply),
        water_supply: Number(waterSupply),
        med_supply: Number(medSupply),
        area_populations: sanitizedPops
      })
    });

    if (!res.ok) {
      const errJson = await res.json().catch(() => ({}));
      throw new Error(errJson.detail || "Failed to execute AI optimization pipeline");
    }

    const json = await res.json();
    setAnalysisData(json);
    setHasUnsavedChanges(false);

    if (navigateToStep2) {
      setActiveStep("2 AI analysis");
    }
  } catch (err) {
    console.error(err);
    setError(`Backend Error: ${err.message}. Ensure FastAPI server is running on port 8000.`);
  } finally {
    setLoading(false);
  }
};
```